

Time Series Classification for 연도별 trend 반영

- 2025년도에 상장폐지될 확률? $\Rightarrow$ sliding window 사용. window size = k (tuning parameter, 5 사용.)
- N-k, ..., N-1 recent k개 년도 data $\rightarrow$ N 년도 상장폐지 확률?
- each sample = (N-k, ..., N-1 recent k개 년도 input feature, N 년도 상장폐지 label)
- dataset = all samples, 기업별 구분 무시 (k개 년도에 따른 label값만 중요)
 - $\Rightarrow$ NA처리 후 기업별 row 개수 차이 무시.

구분	예측 연도 구간	입력 연도 구간 (k = 5 기준)
Train	2004 ~ 2016	1999 ~ 2015
Validation	2017 ~ 2020	2012 ~ 2019
Test	2021 ~ 2024	2016 ~ 2023

$\Rightarrow$ Data가 적은 것 같아서 "<https://gmnam.tistory.com/230>"에 나오는 TimeSeriesSplit 방식 사용했습니다. (time series data의 5folds 방식, 시간의 연속성 때문) blocked cross-validation 의 경우 dataset이 더 많을 때 쓰는 방식같아서 제외했어요

실제 예측 (2025 상장폐지 예측)

- 2020~2024년 재무정보 (최근 5년)를 이용하여 2025년 상장폐지 확률 예측
- 학습된 모델에 해당 윈도우만 inference용으로 입력
- 실제 y-label은 없음 $\rightarrow$ 예측값 (확률)만 산출

■ 상장폐지 중 비 경영악화 요소는 우선 반영하지 않았습니다.

Models

- MLP
- GRU
- TCN
- Ensemble

전처리

1. feature selection

- ratio 처리
- 시계열 data: 회계년 변수 필요
- target feature 관련 변수 포함
 - Symbol(기업 구분용), 상장폐지일자

2. 기업별로 each row마다 inf & NA 처리

- inf

- NA가 적은(10% 미만) row 생존
 - NA가 많은(30% 이상) row 제거
 - NA가 애매한(10%~30%) row는 선형 보간법 사용.
3. 기업별로 첫 상장폐지 이후 row 제거 [회생하는 경우 배제]
4. k개 년도 연속 sliding window 생성
- 불연속 window 제거: 연도가 연속적이지 않은 경우에는 해당 sample 제거. (ex. 2000, **2001**, **2003**, 2004, 2005)
 - each sample마다 y-label 부여.
 - 0: 상장유지, 1: 상장폐지

Input Features Selection @ 홍인택님 추후 수정

- 연도 (회계년) => 결산월은 무시. (12월 결산 대다수, but 일부 3월, 6월 존재.)
- 금융비용/부채
- 금융비용/총비용
- 설비투자효율 = 매출액/유형자산
- 현금흐름/총자본
- 영업이익/총자본
- 유보이익/총자본
- 자기자본비율 = 자기자본/총자산
- 자본금회전율 = 매출액/자본금
- 차입금의존도 = 차입금/총자산
- 총부채회전율 = 매출액/총부채
- 총자본투자효율 = 영업이익/총자산
- 총자본회전율 = 매출액/총자산
- 총순자산수익률 = 당기순이익/자기자본
- 현금성자산비율 = 현금및현금성자산/총자산

```
In [ ]: ! pip install pandas
        ! pip install torch
        ! pip install scikit-learn
```

```
In [1]: import pandas as pd
import numpy as np

import random
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset
from sklearn.model_selection import TimeSeriesSplit
from sklearn.metrics import accuracy_score, f1_score
```

1. main_Dataset_mod.csv Preview

```
In [2]: df = pd.read_csv('main_dataset_mod.csv')

print(df.shape)
```

```
print(df.head())  
print(df.columns.to_list())
```

C:\Users\Public\Documents\ESTsoft\CreatorTemp\ipykernel_17500\1713723279.py:1: DtypeWarning: Columns (260,261) have mixed types. Specify dtype option on import or set low_memory=False.

```
df = pd.read_csv('main_dataset_mod.csv')
```

(163046, 265)

	Symbol	Name	결산월	회계년	주기	총자산(천원)	총자산(평균)(천원)
	유동자산(천원) \						
0	A000010	조흥은행	12.0	1999	Annual	4.693918e+10	4.674953e+10
1	A000010	조흥은행	12.0	2000	Annual	5.113869e+10	4.903894e+10
2	A000010	조흥은행	12.0	2001	Annual	5.661012e+10	5.387440e+10
3	A000010	조흥은행	12.0	2002	Annual	6.727043e+10	6.194027e+10
4	A000010	조흥은행	12.0	2003	Annual	6.060160e+10	6.393601e+10

	당좌자산(천원) 현금및현금성자산(천원) ... 현금흐름(연율화)(천원) 현금흐름(직전4분기)(천원) 현금흐름(보통)(천원) \					
0	NaN	1.210399e+09	...	-746322000.0	NaN	-746322000.0
1	NaN	9.738450e+08	...	99359000.0	NaN	99359000.0
2	NaN	1.346084e+09	...	710634000.0	NaN	710634000.0
3	NaN	1.174976e+09	...	-395695000.0	NaN	-395695000.0
4	NaN	9.181470e+08	...	-750930000.0	NaN	-750930000.0

	현금흐름(보통,연율화)(천원) 현금흐름(보통,직전4분기)(천원)			상장
	폐지일자 \			
0	-746322000.0	NaN	2004-07-02T00:00:00Z	
1	99359000.0	NaN	2004-07-02T00:00:00Z	
2	710634000.0	NaN	2004-07-02T00:00:00Z	
3	-395695000.0	NaN	2004-07-02T00:00:00Z	
4	-750930000.0	NaN	2004-07-02T00:00:00Z	

	폐지사유	파산여부	부도여부	Bad
0	지주회사의 완전자회사화(지주회사 신규상장)	0	0	0
1	지주회사의 완전자회사화(지주회사 신규상장)	0	0	0
2	지주회사의 완전자회사화(지주회사 신규상장)	0	0	0
3	지주회사의 완전자회사화(지주회사 신규상장)	0	0	0
4	지주회사의 완전자회사화(지주회사 신규상장)	0	0	0

[5 rows x 265 columns]

['Symbol', 'Name', '결산월', '회계년', '주기', '총자산(천원)', '총자산(평균)(천원)', '유동자산(천원)', '당좌자산(천원)', '현금및현금성자산(천원)', '단기금융자산(천원)', '단기금융상품(천원)', '단기유가증권(천원)', '단기대여금(천원)', '매출채권및기타채권(천원)', '매출채권(천원)', '단기미청구공사(천원)', '재고자산(천원)', '당기법인세자산(천원)', '매각예정비유동자산(천원)', '비유동자산(천원)', '장기금융자산(천원)', '장기금융상품(천원)', '매도가능금융자산(천원)', '만기보유금융자산(천원)', '장기대여금(천원)', '투자부동산(천원)', '관계기업등지분관련투자자산(천원)', '종속기업투자(천원)', '관계기업투자등(천원)', '유형자산(천원)', '토지(천원)', '설비자산(천원)', '기타유형자산(천원)', '건설중인자산(천원)', '무형자산(천원)', '영업권(천원)', '회원권및제이용권(천원)', '개발비(천원)', '산업재산권(천원)', '비유동생물자산(천원)', '장기매출채권및기타채권(천원)', '이연법인세자산(천원)', '기타금융업자산(천원)', '*단기유동성(천원)', '*총금융자산(천원)', '*총유가증권(천원)', '부채및자본총계(천원)', '총부채(천원)', '총부채(평균)(천원)', '유동부채(천원)', '매입채무및기타채무(천원)', '매입채무(천원)', '초과청구공사(천원)', '단기금융부채(천원)', '단기사채(천원)', '단기차입금(천원)', '유동성장기부채(천원)', '단기총당부채(천원)', '당기법인세부채(천원)', '매각예정처분자산집단부채(천원)', '비유동부채(천원)', '장기금융부채(천원)', '사채(천원)', '일반사채(천원)', '전환사채(천원)', '신주인수권부사채(천원)', '교환사채(천원)', '장기차입금(천원)', '금융리스부채(천원)', '장기매입채무및기타채무(천원)', '비유동종업원급여충당부채(천원)', '기타장기충당부채(천원)', '이연법인세부채(천원)', '기타금융업부채(천원)', '*총금융부채(천원)', '*총차입부채(천원)', '*단기차입부채(천원)', '*장기차입부채(천원)', '*순차입부채(천원)', '*총사채(천원)', '*총금융리스부채(천원)', '총자본(천원)', '총자본(평균)(천원)', '지배주주지분(천원)', '지배주주지분(평균)(천원)', '지배주주순자산(자사주차감전)(천원)', '자본금(천원)', '보통주자본금(천원)', '우선주자본금(천원)', '자본금(평균)(천원)', '신종자본증권(천원)', '자본잉여금(천원)', '주식발행초과금(천원)', '자기주식처분이익(천원)', '기타자본잉여금(천원)', '기타자본구성요소(천원)', '자기주식(천원)', '자기주식처분손실(천원)', '이익잉여금(천원)', '기타포괄손익누계액(천원)', '금융자산평

가손익(천원)', '매도가능금융자산평가손익(천원)', '관계기업등기타포괄손익(천원)',
 '환산관련외환차이적립금(환율변동차이)(천원)', '재평가잉여금(천원)', '비지배주주지
 분(천원)', '*유보액(천원)', '*유보액(3년평균)(천원)', '매출액(천원)', '매출액(3년
 평균)(천원)', '매출액(연율화)(천원)', '매출액(직전4분기)(천원)', '매출원가(천원)',
 '매출총이익(천원)', '판매비와관리비(천원)', '인건비(천원)', '급여(천원)', '퇴직급
 여(천원)', '복리후생비(천원)', '유무형자산상각비(천원)...121', '유형자산감가상각비
 (천원)...122', '무형자산상각비(천원)...123', '연구개발비(천원)', '광고선전비(천
 원)', '판매비(천원)', '대손상각비(판관비)(천원)', '관리비(천원)', '임차료(천원)',
 '세금과공과(천원)', '영업이익(천원)', '영업이익(3년평균)(천원)', '영업이익(연율화)
 (천원)', '영업이익(직전4분기)(천원)', '*조정영업이익(천원)', '*조정영업이익(직전4
 분기)(천원)', '비영업손익(천원)', '금융손익(비영업)(천원)', '관계기업투자등관련손
 익(비영업)(천원)', '기타손익(비영업)(천원)', '외환관련손익(비영업)(천원)', '자산재
 평가손익(천원)', '비영업수익(천원)', '금융수익(비영업)(천원)', '관계기업투자등관련
 이익(비영업)(천원)', '기타수익(비영업)(천원)', '이자수익(비영업)(천원)', '배당금수
 익(비영업)(천원)', '외환이익(비영업)(천원)', '외환차익(비영업)(천원)', '외환환산이
 익(비영업)(천원)', '유가증권관련이익(비영업)(천원)', '비영업비용(천원)', '금융원가
 (비영업)(천원)', '관계기업투자등관련손실(비영업)(천원)', '기타비용(비영업)(천원)',
 '이자비용(비영업)(천원)', '*순이자비용(비영업)(천원)', '외환손실(비영업)(천원)',
 '외환차손(비영업)(천원)', '외화환산손실(비영업)(천원)', '유가증권관련손실(비영업)
 (천원)', '세전계속사업이익(천원)', '세전계속사업이익(3년평균)(천원)', '세전계속사
 업이익(연율화)(천원)', '세전계속사업이익(직전4분기)(천원)', '계속사업법인세비용(천
 원)', '계속사업이익(천원)', '계속사업이익(직전4분기)(천원)', '중단사업이익(천원)',
 '중단사업이익(직전4분기)(천원)', '당기순이익(천원)', '당기순이익(3년평균)(천원)',
 '당기순이익(연율화)(천원)', '당기순이익(직전4분기)(천원)', '지배주주순이익(천원)',
 '지배주주순이익(3년평균)(천원)', '지배주주순이익(연율화)(천원)', '지배주주순이익
 (직전4분기)(천원)', '비지배주주순이익(천원)', '기타포괄손익(천원)', '총포괄이익(천
 원)', '지배주주총포괄이익(천원)', '비지배주주총포괄이익(천원)', '순영업이익(천
 원)', '영업수익(천원)', '영업수익(3년평균)(천원)', '영업수익(연율화)(천원)', '영업
 수익(직전4분기)(천원)', '영업비용(천원)', '영업활동으로인한현금흐름(천원)', '영업
 활동으로인한현금흐름(직전4분기)(천원)', '영업활동으로인한현금흐름(평균)(천원)',
 '당기순손익(법인세차감전순손익)(천원)', '현금유출입이없는비용(수익)(천원)', '현금
 유출이없는비용의가산(천원)', '유무형자산상각비(천원)...197', '유형자산감가상각비
 (천원)...198', '무형자산상각비(천원)...199', '이자비용(천원)', '외환손실(천원)',
 '유가증권관련손실(천원)', '관계회사등투자손실(천원)', '법인세비용(천원)', '외화환
 산손실(이익)(천원)', '현금유입이없는수익의차감(천원)', '이자수익(천원)', '외환이
 익(천원)', '유가증권관련이익(천원)', '관계회사등투자이익(천원)', '영업활동으로인한
 자산및부채의변동(천원)', '영업활동으로인한자산의감소(천원)', '매출채권및기타채권의
 감소(증가)(천원)', '재고자산의감소(증가)(천원)', '영업활동으로인한부채의증가(천
 원)', '매입채무및기타채무의증가(감소)(천원)', '기타영업자산및부채의변동(천원)',
 '투자활동으로인한현금흐름(천원)', '투자활동으로인한현금유입액(천원)', '투자활동으
 로인한현금유출액(천원)', '유형자산처분(취득)(천원)', '무형자산감소(증가)(천원)',
 '*유형자산순취득액(천원)', '*무형자산순취득액(천원)', '장단기금융자산의감소(증가)
 (천원)', '단기금융자산의감소(증가)(천원)', '장기금융자산의감소(증가)(천원)', '기타
 투자활동으로인한현금흐름(천원)', '재무활동으로인한현금흐름(천원)', '재무활동으로인
 한현금유입액(천원)', '장단기금융부채의증가(천원)', '자본의증가(천원)', '재무활동으
 로인한현금유출액(천원)', '장단기금융부채의감소(천원)', '자본의감소(천원)', '장단기
 금융부채의증가(감소)(천원)', '단기금융부채의증가(감소)(천원)', '장기금융부채의증가
 (감소)(천원)', '자본의증가(감소)(천원)', '기타재무활동으로인한현금흐름(천원)', '이
 자수취(영업,투자,재무)(천원)', '이자지급(영업,투자,재무)(천원)', '배당금수취(영업,
 투자,재무)(천원)', '배당금지급(영업,투자,재무)(천원)', '법인세환급(영업,투자,재무)
 (천원)', '법인세납부(영업,투자,재무)(천원)', '기타현금흐름(천원)', '영업,투자,재무
 활동기타현금흐름(천원)', '연결범위변동으로인한현금의증가(천원)', '환율변동효과(천
 원)', '합병분할(영업양수도)으로인한증가(감소)(천원)', '현금의순증가(천원)', '기초
 현금(천원)', '기말현금(천원)', '현금흐름(천원)', '현금흐름(연율화)(천원)', '현금흐
 림(직전4분기)(천원)', '현금흐름(보통)(천원)', '현금흐름(보통,연율화)(천원)', '현금
 흐름(보통,직전4분기)(천원)', '상장폐지일자', '폐지사유', '파산여부', '부도여부',
 'Bad']

2. 전처리

- Feature Selection

```
In [3]: df["연도"] = df["회계년"].astype(int)
df["상장폐지년도"] = pd.to_datetime(df["상장폐지일자"], errors="coerce").dt.year

# 파생변수 생성
df['금융비용/부채'] = df['이자비용(비영업)(천원)'] / df['총부채(천원)']

total_cost = df['매출원가(천원)'] + df['판매비와관리비(천원)']
df['금융비용/총비용'] = df['이자비용(비영업)(천원)'] / total_cost

df['설비투자효율'] = df['매출액(천원)'] / df['유형자산(천원)']

df['현금흐름/총자본'] = df['영업활동으로인한현금흐름(천원)'] / df['총자본(천원)']

df['영업이익/총자본'] = df['영업이익(천원)'] / df['총자본(천원)']

df['유보이익/총자본'] = df['이익잉여금(천원)'] / df['총자본(천원)']

df['자기자본비율'] = df['총자본(천원)'] / df['총자산(천원)']

df['자본금회전율'] = df['매출액(천원)'] / df['자본금(천원)']

total_loans = df['단기차입금(천원)'] + df['장기차입금(천원)']
df['차입금의존도'] = total_loans / df['총자산(천원)']

df['총부채회전율'] = df['매출액(천원)'] / df['총부채(천원)']

df['총자본투자효율'] = df['영업이익(천원)'] / df['총자산(천원)']

df['총자본회전율'] = df['매출액(천원)'] / df['총자산(천원)']

df['총순자산수익률'] = df['당기순이익(천원)'] / df['총자본(천원)']

df['현금성자산비율'] = df['현금및현금성자산(천원)'] / df['총자산(천원)']

selected_columns = [
    "Symbol", "연도", "상장폐지년도",
    '금융비용/부채', '금융비용/총비용', '설비투자효율', '현금흐름/총자본',
    '영업이익/총자본', '유보이익/총자본', '자기자본비율', '자본금회전율',
    '차입금의존도', '총부채회전율', '총자본투자효율', '총자본회전율',
    '총순자산수익률', '현금성자산비율'
]

df = df[selected_columns].copy()
```

- inf & NA 처리

```
In [11]: def process_missing_and_inf(group):
    group = group.replace([np.inf, -np.inf], np.nan)
    row_na_ratio = group.isna().mean(axis=1)

    cond_low = row_na_ratio < 0.10
    cond_mid = (row_na_ratio >= 0.10) & (row_na_ratio < 0.30)
    cond_high = row_na_ratio >= 0.30
```

```

group = group[~cond_high] # 제거
group.loc[cond_mid] = group.loc[cond_mid].interpolate(method='linear', limit

return group.dropna()

processed_groups = []

for symbol, group in df.groupby('Symbol'):
    processed_group = process_missing_and_inf(group)
    if not processed_group.empty:
        processed_groups.append(processed_group)

df = pd.concat(processed_groups, ignore_index=True)

```

- 상장폐지 이후 row 제거

```

In [12]: def remove_after_delisting(group):
    if group['상장폐지년도'].notna().any():
        first_delist_year = group['상장폐지년도'].dropna().iloc[0]
        return group[group['연도'] <= first_delist_year]
    return group

processed_groups = []

for symbol, group in df.groupby('Symbol'):
    processed_group = remove_after_delisting(group)
    if not processed_group.empty:
        processed_groups.append(processed_group)

df = pd.concat(processed_groups, ignore_index=True)

```

- k개 년도 sliding window 생성

```

In [14]: def generate_sliding_windows(df, k):
    samples = []
    grouped = df.groupby('Symbol')

    for symbol, group in grouped:
        group = group.sort_values('연도')
        years = group['연도'].values
        features = group.drop(columns=['Symbol', '상장폐지년도']).values

        for i in range(len(group) - k):
            window_years = years[i:i + k + 1]
            if not np.all(np.diff(window_years) == 1): # 연속 연도 아닌 경우 제
                continue

            X_window = features[i:i + k] # k개 년도의 연속된 feature
            prediction_year = years[i + k] # 예측할 연도
            y_delist_year = group.iloc[i + k]['상장폐지년도'] # 실제 상장폐지년도

            # 상장폐지년도 = 예측년도 or 예측년도 + 1 ... 재무재표가 너무 안좋아서
            if pd.notna(y_delist_year) and (y_delist_year == prediction_year or
                y_label = 1
            else:

```

```

        y_label = 0

        samples.append((X_window, y_label, prediction_year))

    return samples

k = 3 # tuning parameter, k=5는 data가 너무 적어서
samples = generate_sliding_windows(df, k) # X_window, y_label, prediction_year

# prediction_year 기준으로 정렬
samples.sort(key=lambda x: x[2])

```

- y-label distribution

```

In [15]: def check_label_distribution(dataset, name="Dataset"):
        # (X, y) 또는 (X, y, year) 모두 처리 가능
        labels = [y for item in dataset for i, y in enumerate(item) if i == 1]
        unique, counts = np.unique(labels, return_counts=True)

        print(f"\n{name} 샘플 수: {len(labels)}")
        for u, c in zip(unique, counts):
            print(f"- Label {u}: {c}개 ({c / len(labels):.2%}")

        check_label_distribution(samples)

```

Dataset 샘플 수: 8069
 - Label 0: 6718개 (83.26%)
 - Label 1: 1351개 (16.74%)

- sample: X_window, y_label, prediction_year

```

In [18]: print(f"1st sample example: \n{samples[0]}\n")

        print(f"1st sample input: {samples[0][0]}")
        print(f"1st sample label: {samples[0][1]}")
        if samples[0][1] == 1:
            print(f"1st sample 상장폐지년도: {samples[0][2]}")
        else:
            print(f"1st sample: 상장폐지 X")
        print(f"1st sample 입력연도: {samples[0][2]-5}-{samples[0][2]-1}")
        print(f"1st sample 예측연도: {samples[0][2]}")

        print([s[2] for s in samples[:10]]) # 처음 10개 예측 연도 출력

```

1st sample example:

```
(array([[ 1.99900000e+03,  8.35109894e-02,  2.78140471e-01,
         1.74957211e+00,  4.67534062e-01,  2.68175846e-01,
        -6.73393307e-01,  4.33850341e-02,  9.08179688e+00,
         6.47113490e-01,  3.12410018e-01,  1.16348182e-02,
         2.98856099e-01, -6.86382413e-01,  2.76861752e-02],
       [ 2.00000000e+03,  1.30029729e-01,  5.61038888e-01,
         1.28152799e+00,  8.92760649e-01,  1.77148473e+00,
        -3.98754226e-02,  3.94852487e-02,  7.55933735e+00,
         7.39145534e-01,  3.04588929e-01,  6.99475152e-02,
         2.92562160e-01,  1.12838455e+00,  0.00000000e+00],
       [ 2.00100000e+03,  1.10252464e-01,  3.18289532e-01,
         2.03078422e+00, -6.81854482e-01, -8.52315653e-01,
         1.47917529e+00, -7.16378478e-02,  1.17165341e+01,
         7.68578074e-01,  4.03366873e-01,  6.10580590e-02,
         4.32263208e-01,  1.23246141e+00,  0.00000000e+00]]), 1, np.int64(2002))
```

1st sample input: [[1.99900000e+03 8.35109894e-02 2.78140471e-01 1.74957211e+00

```
  4.67534062e-01  2.68175846e-01 -6.73393307e-01  4.33850341e-02
  9.08179688e+00  6.47113490e-01  3.12410018e-01  1.16348182e-02
  2.98856099e-01 -6.86382413e-01  2.76861752e-02]
[ 2.00000000e+03  1.30029729e-01  5.61038888e-01  1.28152799e+00
  8.92760649e-01  1.77148473e+00 -3.98754226e-02  3.94852487e-02
  7.55933735e+00  7.39145534e-01  3.04588929e-01  6.99475152e-02
  2.92562160e-01  1.12838455e+00  0.00000000e+00]
[ 2.00100000e+03  1.10252464e-01  3.18289532e-01  2.03078422e+00
 -6.81854482e-01 -8.52315653e-01  1.47917529e+00 -7.16378478e-02
  1.17165341e+01  7.68578074e-01  4.03366873e-01  6.10580590e-02
  4.32263208e-01  1.23246141e+00  0.00000000e+00]]
```

1st sample label: 1

1st sample 상장폐지년도: 2002

1st sample 입력연도: 1997-2001

1st sample 예측연도: 2002

```
[np.int64(2002), np.int64(2002), np.int64(2002), np.int64(2002), np.int64(2002),
 np.int64(2002), np.int64(2002), np.int64(2002), np.int64(2002), np.int64(2002)]
```

- 배열로 변환: TimeSeriesSplit 위험
- Test set 분리: 2021~2024년

```
In [23]: X_window = np.array([s[0] for s in samples])
y_label = np.array([s[1] for s in samples])
prediction_year = np.array([s[2] for s in samples])

test_mask = (prediction_year >= 2021)
X_test = X_window[test_mask]
y_test = y_label[test_mask]

X_modeling = X_window[~test_mask]
y_modeling = y_label[~test_mask]
```

- Case sampling: y label ratio 1:1
- TimeSeriesSplit 도중 weight를 부여해서 비중을 1:1로 맞추 예정이에요.

```
In [25]: def check_label_distribution_from_array(y, name=""):
y = np.array(y)
```

```

total = len(y)
num_1 = np.sum(y == 1)
num_0 = np.sum(y == 0)
print(f"[{name}] Total: {total}, 1 비율: {num_1/total:.4f}, 1 개수: {num_1},

check_label_distribution_from_array(y_modeling, "Modeling Set")
check_label_distribution_from_array(y_test, "Test Set")

```

[Modeling Set] Total: 7708, 1 비율: 0.1472, 1 개수: 1135, 0 개수: 6573

[Test Set] Total: 361, 1 비율: 0.5983, 1 개수: 216, 0 개수: 145

Test Set에서 covid19 때문인지 상장폐지 기업이 많네요.

3. Modeling

- MLP

```

In [26]: class MLP(nn.Module):
          def __init__(self, input_size):
              super().__init__()
              self.fc = nn.Sequential(
                  nn.Flatten(),
                  nn.Linear(input_size, 128),
                  nn.BatchNorm1d(128),
                  nn.ReLU(),
                  nn.Dropout(0.3),
                  nn.Linear(128, 64),
                  nn.BatchNorm1d(64),
                  nn.ReLU(),
                  nn.Dropout(0.3),
                  nn.Linear(64, 1)
              )

          def forward(self, x):
              return self.fc(x)

# hyperparameters
n_splits = 5
batch_size = 64
lr = 1e-4
num_epochs = 100

input_size = X_modeling.shape[1] * X_modeling.shape[2]

```

Cross Validation: MLP

- accuracy와 함께 F1-score를 사용했습니다. (unbalanced ratio data)

```

In [38]: prediction_year_modeling = prediction_year[~(prediction_year >= 2021)]
          assert len(prediction_year_modeling) == len(X_modeling)

          tscv = TimeSeriesSplit(n_splits=n_splits)

```

```

print("==== TimeSeriesSplit MLP Training with Weighted Loss ====")
for fold, (train_idx, val_idx) in enumerate(tscv.split(X_modeling)):
    # Fold별 train/val 데이터 준비
    X_train = torch.tensor(X_modeling[train_idx], dtype=torch.float32)
    y_train = torch.tensor(y_modeling[train_idx], dtype=torch.float32)
    X_val = torch.tensor(X_modeling[val_idx], dtype=torch.float32)
    y_val = torch.tensor(y_modeling[val_idx], dtype=torch.float32)

    # 연도 정보도 인덱싱 일관성 있게
    train_years = prediction_year_modeling[train_idx]
    val_years = prediction_year_modeling[val_idx]
    print(f"[Fold {fold+1}] 예측 연도 범위 (Train): {train_years.min()} ~ {train_years.max()}")

    # 가중치 계산 (positive label 보정)
    num_pos = (y_train == 1).sum().item()
    num_neg = (y_train == 0).sum().item()
    pos_weight = num_neg / num_pos if num_pos > 0 else 1.0
    pos_weight_tensor = torch.tensor([pos_weight], dtype=torch.float32)

    # 모델, 손실함수, 옵티마이저 정의
    model = MLP(input_size)
    criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight_tensor)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    # DataLoader
    train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=batch_size)

    # Training
    for epoch in range(1, num_epochs + 1):
        model.train()
        total_loss = 0
        for xb, yb in train_loader:
            optimizer.zero_grad()
            logits = model(xb).squeeze()
            loss = criterion(logits, yb)
            loss.backward()
            optimizer.step()
            total_loss += loss.item()

    # Validation
    model.eval()
    with torch.no_grad():
        val_logits = model(X_val).squeeze()
        val_probs = torch.sigmoid(val_logits).numpy()
        val_pred = (val_probs >= 0.5).astype(int)
        acc = accuracy_score(y_val.numpy(), val_pred)
        f1 = f1_score(y_val.numpy(), val_pred, zero_division=0)
        print(f"[Fold {fold+1}] Validation Accuracy: {acc:.4f}, F1 Score: {f1:.4f}")

```

```

==== TimeSeriesSplit MLP Training with Weighted Loss ====
[Fold 1] 예측 연도 범위 (Train): 2002 ~ 2004, (Val): 2004 ~ 2006
[Fold 1] Validation Accuracy: 0.1090, F1 Score: 0.1320

[Fold 2] 예측 연도 범위 (Train): 2002 ~ 2006, (Val): 2006 ~ 2008
[Fold 2] Validation Accuracy: 0.1815, F1 Score: 0.1859

[Fold 3] 예측 연도 범위 (Train): 2002 ~ 2008, (Val): 2008 ~ 2011
[Fold 3] Validation Accuracy: 0.2391, F1 Score: 0.3285

[Fold 4] 예측 연도 범위 (Train): 2002 ~ 2011, (Val): 2011 ~ 2015
[Fold 4] Validation Accuracy: 0.5202, F1 Score: 0.3186

[Fold 5] 예측 연도 범위 (Train): 2002 ~ 2015, (Val): 2015 ~ 2020
[Fold 5] Validation Accuracy: 0.7671, F1 Score: 0.0000

```

- GRU

```

In [ ]: class GRUModel(nn.Module):
    def __init__(self, input_size, hidden_size=64, num_layers=1, bidirectional=False):
        super().__init__()
        self.gru = nn.GRU(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True,
            bidirectional=bidirectional
        )
        self.fc = nn.Linear(hidden_size * (2 if bidirectional else 1), 1)

    def forward(self, x):
        # x: (batch, seq_len, input_size)
        out, _ = self.gru(x)
        # 마지막 타임스텝의 출력만 사용
        out = out[:, -1, :] # shape: (batch, hidden_size)
        return self.fc(out) # Logits (no sigmoid)

# hyperparameters
n_splits = 5
batch_size = 64
lr = 1e-4
num_epochs = 30

input_dim = X_modeling.shape[2] # 시계열 feature 수

```

```

In [32]: # 연도 정렬 가정 하에 modeling dataset 추출
prediction_year_modeling = prediction_year[~(prediction_year >= 2021)]
assert len(prediction_year_modeling) == len(X_modeling)

tscv = TimeSeriesSplit(n_splits=n_splits)

print("==== TimeSeriesSplit GRU Training with Weighted Loss ====")
for fold, (train_idx, val_idx) in enumerate(tscv.split(X_modeling)):
    # Fold별 데이터 준비

```

```

X_train = torch.tensor(X_modeling[train_idx], dtype=torch.float32)
y_train = torch.tensor(y_modeling[train_idx], dtype=torch.float32)
X_val = torch.tensor(X_modeling[val_idx], dtype=torch.float32)
y_val = torch.tensor(y_modeling[val_idx], dtype=torch.float32)

# 연도 출력
train_years = prediction_year_modeling[train_idx]
val_years = prediction_year_modeling[val_idx]
print(f"[Fold {fold+1}] 입력 연도 범위 (Train): {train_years.min()} ~ {train_years.max()} (Val): {val_years.min()} ~ {val_years.max()}")

# 클래스 불균형 보정
num_pos = (y_train == 1).sum().item()
num_neg = (y_train == 0).sum().item()
pos_weight = num_neg / num_pos if num_pos > 0 else 1.0
pos_weight_tensor = torch.tensor([pos_weight], dtype=torch.float32)

# 모델 및 학습 설정
model = GRUModel(input_size=X_modeling.shape[2])
criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight_tensor)
optimizer = torch.optim.Adam(model.parameters(), lr=lr)
train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=batch_size)

# 학습 루프
for epoch in range(1, num_epochs + 1):
    model.train()
    total_loss = 0
    for xb, yb in train_loader:
        optimizer.zero_grad()
        output = model(xb).squeeze()
        loss = criterion(output, yb)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

# 검증
model.eval()
with torch.no_grad():
    val_logits = model(X_val).squeeze()
    val_probs = torch.sigmoid(val_logits).numpy()
    val_pred = (val_probs >= 0.5).astype(int)
    acc = accuracy_score(y_val.numpy(), val_pred)
    f1 = f1_score(y_val.numpy(), val_pred, zero_division=0)
    print(f"[Fold {fold+1}] Validation Accuracy: {acc:.4f}, F1 Score: {f1:.4f}")

```

==== TimeSeriesSplit GRU Training with Weighted Loss ====

[Fold 1] 입력 연도 범위 (Train): 2002 ~ 2004, (Val): 2004 ~ 2006

[Fold 1] Validation Accuracy: 0.9299, F1 Score: 0.0000

[Fold 2] 입력 연도 범위 (Train): 2002 ~ 2006, (Val): 2006 ~ 2008

[Fold 2] Validation Accuracy: 0.8069, F1 Score: 0.0815

[Fold 3] 입력 연도 범위 (Train): 2002 ~ 2008, (Val): 2008 ~ 2011

[Fold 3] Validation Accuracy: 0.7936, F1 Score: 0.0569

[Fold 4] 입력 연도 범위 (Train): 2002 ~ 2011, (Val): 2011 ~ 2015

[Fold 4] Validation Accuracy: 0.8364, F1 Score: 0.0625

[Fold 5] 입력 연도 범위 (Train): 2002 ~ 2015, (Val): 2015 ~ 2020

[Fold 5] Validation Accuracy: 0.2967, F1 Score: 0.3785

- TCN

```
In [34]: class Chomp1d(nn.Module):
    """Conv1d의 padding 때문에 생기는 output 뒤쪽 값을 잘라내는 역할"""
    def __init__(self, chomp_size):
        super().__init__()
        self.chomp_size = chomp_size

    def forward(self, x):
        return x[:, :, :-self.chomp_size]

class TemporalBlock(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, stride, dilation,
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv1d(in_channels, out_channels, kernel_size, stride=stride,
                padding=padding, dilation=dilation),
            Chomp1d(padding),
            nn.BatchNorm1d(out_channels),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Conv1d(out_channels, out_channels, kernel_size, stride=stride,
                padding=padding, dilation=dilation),
            Chomp1d(padding),
            nn.BatchNorm1d(out_channels),
            nn.ReLU(),
            nn.Dropout(dropout)
        )
        self.downsample = nn.Conv1d(in_channels, out_channels, 1) if in_channels
        self.relu = nn.ReLU()

    def forward(self, x):
        out = self.net(x)
        res = x if self.downsample is None else self.downsample(x)
        return self.relu(out + res)

class TCN(nn.Module):
    def __init__(self, input_size, num_channels, kernel_size=2, dropout=0.2):
        super().__init__()
        layers = []
        num_levels = len(num_channels)
        for i in range(num_levels):
            dilation_size = 2 ** i
            in_ch = input_size if i == 0 else num_channels[i - 1]
            out_ch = num_channels[i]
            layers += [TemporalBlock(in_ch, out_ch, kernel_size, stride=1,
                dilation=dilation_size,
                padding=(kernel_size - 1) * dilation_size,
                dropout=dropout)]

        self.tcn = nn.Sequential(*layers)
        self.fc = nn.Linear(num_channels[-1], 1)

    def forward(self, x):
        # x: (batch, seq_len, input_dim)
        x = x.permute(0, 2, 1) # → (batch, input_dim, seq_len)
        y = self.tcn(x)        # → (batch, out_ch, seq_len)
```

```

y = y[:, :, -1]          # 마지막 타임스텝만
return self.fc(y)        # logits 출력

```

```

In [35]: # 연도 정렬 가정 하에 modeling dataset 추출
prediction_year_modeling = prediction_year[~(prediction_year >= 2021)]
assert len(prediction_year_modeling) == len(X_modeling)

tscv = TimeSeriesSplit(n_splits=n_splits)

print("==== TimeSeriesSplit TCN Training with Weighted Loss ====")
for fold, (train_idx, val_idx) in enumerate(tscv.split(X_modeling)):
    # FoLd별 데이터
    X_train = torch.tensor(X_modeling[train_idx], dtype=torch.float32) # (batch
    y_train = torch.tensor(y_modeling[train_idx], dtype=torch.float32)
    X_val = torch.tensor(X_modeling[val_idx], dtype=torch.float32)
    y_val = torch.tensor(y_modeling[val_idx], dtype=torch.float32)

    train_years = prediction_year_modeling[train_idx]
    val_years = prediction_year_modeling[val_idx]
    print(f"[Fold {fold+1}] 입력 연도 범위 (Train): {train_years.min()} ~ {train

    # 클래스 비율에 따른 가중치
    num_pos = (y_train == 1).sum().item()
    num_neg = (y_train == 0).sum().item()
    pos_weight = num_neg / num_pos if num_pos > 0 else 1.0
    pos_weight_tensor = torch.tensor([pos_weight], dtype=torch.float32)

    # TCN 모델 정의
    model = TCN(
        input_size=X_modeling.shape[2], # input_dim = feature 수
        num_channels=[64, 64, 64],
        kernel_size=3,
        dropout=0.3
    )
    criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight_tensor)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=batch_

    # 학습 루프
    for epoch in range(1, num_epochs + 1):
        model.train()
        total_loss = 0
        for xb, yb in train_loader:
            optimizer.zero_grad()
            logits = model(xb).squeeze()
            loss = criterion(logits, yb)
            loss.backward()
            optimizer.step()
            total_loss += loss.item()

    # 검증
    model.eval()
    with torch.no_grad():
        val_logits = model(X_val).squeeze()
        val_probs = torch.sigmoid(val_logits).numpy()
        val_pred = (val_probs >= 0.5).astype(int)
        acc = accuracy_score(y_val.numpy(), val_pred)

```

```
f1 = f1_score(y_val.numpy(), val_pred, zero_division=0)
print(f"[Fold {fold+1}] Validation Accuracy: {acc:.4f}, F1 Score: {f1:.4f}")
```

```
==== TimeSeriesSplit TCN Training with Weighted Loss ====
[Fold 1] 입력 연도 범위 (Train): 2002 ~ 2004, (Val): 2004 ~ 2006
[Fold 1] Validation Accuracy: 0.8956, F1 Score: 0.1519

[Fold 2] 입력 연도 범위 (Train): 2002 ~ 2006, (Val): 2006 ~ 2008
[Fold 2] Validation Accuracy: 0.8956, F1 Score: 0.0429

[Fold 3] 입력 연도 범위 (Train): 2002 ~ 2008, (Val): 2008 ~ 2011
[Fold 3] Validation Accuracy: 0.2017, F1 Score: 0.3322

[Fold 4] 입력 연도 범위 (Train): 2002 ~ 2011, (Val): 2011 ~ 2015
[Fold 4] Validation Accuracy: 0.4953, F1 Score: 0.2926

[Fold 5] 입력 연도 범위 (Train): 2002 ~ 2015, (Val): 2015 ~ 2020
[Fold 5] Validation Accuracy: 0.2531, F1 Score: 0.3809
```

4. 2025년 상장폐지 예측

```
In [37]: def evaluate_model(model, X, y_true, model_name):
    model.eval()
    with torch.no_grad():
        logits = model(X).squeeze()
        probs = torch.sigmoid(logits)
        preds = (probs >= 0.5).long()
        acc = accuracy_score(y_true.numpy(), preds.numpy())
        f1 = f1_score(y_true.numpy(), preds.numpy(), zero_division=0)
        print(f"[{model_name}] Test Accuracy: {acc:.4f}, F1 Score: {f1:.4f}")

# Tensor 변환
X_test_mlp = torch.tensor(X_test, dtype=torch.float32) # (N, input_dim)
X_test_seq = torch.tensor(X_test, dtype=torch.float32) # (N, seq_len)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32)

model_mlp = MLP(X_modeling.shape[1] * X_modeling.shape[2])
model_gru = GRUModel(X_modeling.shape[2])
model_tcn = TCN(
    input_size=X_modeling.shape[2], # input_dim = feature 수
    num_channels=[64, 64, 64],
    kernel_size=3,
    dropout=0.3
)

# 평가
evaluate_model(model_mlp, X_test_mlp, y_test_tensor, "MLP")
evaluate_model(model_gru, X_test_seq, y_test_tensor, "GRU")
evaluate_model(model_tcn, X_test_seq, y_test_tensor, "TCN")

[MLP] Test Accuracy: 0.5983, F1 Score: 0.7487
[GRU] Test Accuracy: 0.5706, F1 Score: 0.6856
[TCN] Test Accuracy: 0.4017, F1 Score: 0.0000
```